SPAços

Especificação técnica do banco de dados

Modelo documental • MongoDB • Integração de dados culturais

VERSÃO DE REFERÊNCIA: v3

BASE DOCUMENTAL: arquitetura inicial do SPAços + respostas de Mapas Culturais e GeoSampa

DATA: 29 de outubro de 2026

STATUS: baseline técnica implementável, pendente de validação em servidor MongoDB real

Objetivo

Servir como referência única para compreender as entidades do sistema, os documentos persistidos, os tipos e validações de campos, os relacionamentos, os índices, a origem dos dados e os procedimentos de instalação e operação.

Como ler este documento "Definido no v3" significa que a estrutura foi especificada no script MongoDB anterior. "Validado em amostra" significa que o formato foi observado nos JSON/XML recebidos. Isso não equivale a teste de carga ou execução no servidor MongoDB.

Sumário de leitura

- **01. Escopo e visão geral** — Objetivo, limites e catálogo de coleções
- **02. Decisões estruturais** — Por que MongoDB, referências, documentos incorporados e rastreabilidade
- **03. Relacionamentos** — Cardinalidades e consistência entre coleções
- **04. Dicionário de documentos** — Cada coleção, seus campos, tipos e obrigatoriedade
- **05. Fontes e normalização** — Mapas Culturais, GeoSampa e critérios de qualidade
- **06. Validações e índices** — JSON Schema e desempenho das consultas
- **07. Operação no Windows** — Criação, importação, consultas de conferência e cuidados
- **08. Regras, riscos e evolução** — Pendências para produção e futuras decisões

1. Escopo e visão geral

O SPAços é uma aplicação web/PWA para descobrir espaços culturais e montar roteiros de visitação na cidade de São Paulo. O banco sustenta o mapa interativo, filtros temáticos e de inclusão, programação de eventos, roteiros, contas de usuários, favoritos e histórico de visitas. O backend FastAPI é responsável por consultas, sincronização e aplicação das regras de negócio; o frontend não acessa diretamente as fontes externas.

Este documento descreve o baseline MongoDB v3 construído ao longo da modelagem do projeto, incluindo seus pontos ainda incompletos. Não pressupõe que todos os documentos das APIs sejam atuais, que tenham informações completas ou que a instância MongoDB local já tenha executado os scripts.

1.1. Catálogo das coleções

Coleção	Responsabilidade	Grupo
places	Cadastro consolidado de pontos e espaços culturais, localização, filtros e proveniência	Cultura / domínio
events	Eventos culturais e suas ocorrências (potencialmente em locais e datas diferentes)	Cultura / domínio
territories	Geometrias de distritos e subprefeituras para enriquecimento espacial	Geografia
place_curations	Atributos editoriais verificados, independentes das APIs	Curadoria
sync_runs	Auditoria do processamento e das sincronizações de fontes	Infraestrutura
users	Conta, autenticação e preferências pessoais	Contas
itineraries	Roteiros criados/salvos e paradas ordenadas	Roteiros
user_places	Relação usuário–lugar (favorito e visitado)	Interações

1.2. Dentro e fora da persistência

Persistidos: catálogos culturais sincronizados, geometrias administrativas, curadoria, contas, roteiros e histórico de interações. Consultados sob demanda e com cache de curta duração quando conveniente: trajetos/linhas/estimativas de transporte da SPTrans e previsão meteorológica da Open-Meteo. Não existem coleções persistentes específicas para clima ou viagens no baseline v3.

2. Decisões estruturais

Decisão	Implementação e justificativa
MongoDB documental	Atributos de lugares são heterogêneos e parcialmente preenchidos. O documento concentra o que a página do lugar normalmente consulta.
Dados embutidos	Descrição, endereço, contatos, preço, acessibilidade, fonte e qualidade pertencem ao lugar; ocorrências pertencem ao evento; paradas pertencem ao roteiro.
Referências por identificador	Roteiros, eventos, favoritos e curadoria referenciam places. O MongoDB não cria chaves estrangeiras automaticamente.
Separação entre importação e curadoria	place_curations não é sobrescrita pelas rotinas de sincronização; origem e verificação acompanham os dados.
Identificadores estáveis string	Os `_id` têm formato específico de fonte (ex.: mapas_saas:space:1), enquanto `source_keys[]` pode armazenar várias identidades do mesmo espaço.
Dados desconhecidos ≠ falsos	Preço, recursos físicos, visitas e horários não informados permanecem null ou vazios; ausência não comprova falta de acessibilidade.
GeoJSON + 2dsphere	Pontos culturais em places.location; polígonos administrativos em territories.geometry; coordenadas em longitude, latitude (EPSG:4326).
Publicação controlada	`publication_status` distingue revisão, publicação, ocultação e arquivamento; valor da API em `source_status` não equivale a aprovação pelo SPAços.
Job de sincronização	Pipelines coletam, normalizam, validam, deduplicam quando possível, enriquecem e persistem. `sync_runs` registra auditoria.

2.1. Por que nem tudo vira coleção?

Criar tabelas separadas para horários, acessibilidade, categorias ou paradas de roteiro geraria referências desnecessárias para o conjunto de consultas do MVP. Por isso, o v3 usa subdocumentos. Em contrapartida, o vínculo usuário–lugar é potencialmente muitos-para-muitos e recebe coleção própria (`user_places`); curadoria é independente porque possui ciclo de vida e responsabilidade editorial diferentes.

3. Relacionamentos e cardinalidades

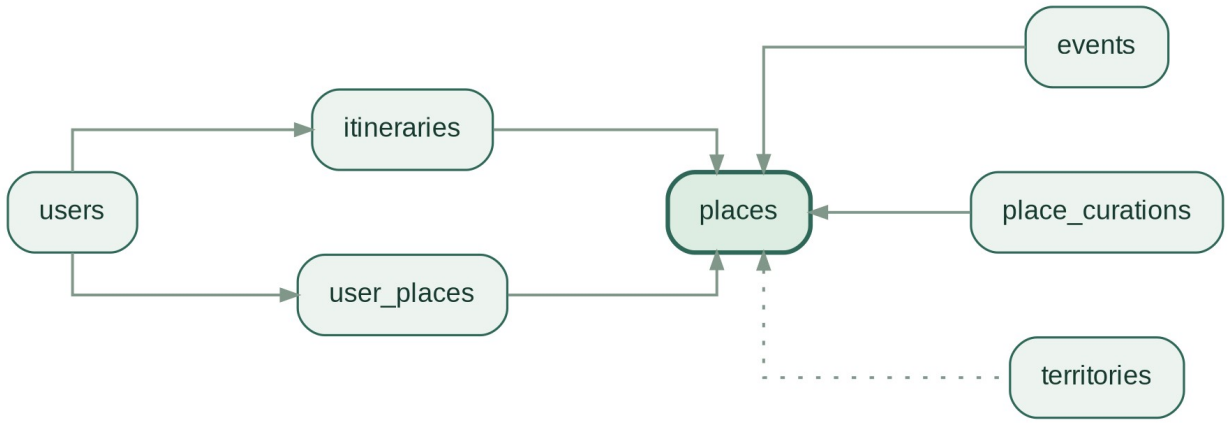


Figura 1. Relações de domínio (a coleção sync_runs é independente e controla a integração).

Origem	Destino	Representação no MongoDB
users (1) → itineraries (N)	itineraries	Campo `itineraries.user_id` = `users._id`.
users (1) → user_places (N)	user_places	Campo `user_places.user_id` = `users._id`.
places (1) → user_places (N)	user_places	Campo `user_places.place_id` = `places._id`.
itineraries (N) ↔ places (N)	places	Array incorporado `itineraries.stops[].place_id`.
events (N) ↔ places (N)	places	Array `events.occurrences[].place_id`; um evento pode ocorrer em múltiplos espaços.
places (1) ↔ place_curations (0..1)	place_curations	Campo `place_curations.place_id` com índice único.
territories ↔ places	places	Correspondência geoespacial, não chave estrangeira declarada no v3.
sync_runs → fontes importadas	places / events	Relação operacional de auditoria; sem foreign key formal.

Integridade referencial As referências são strings e não são validadas automaticamente como chaves estrangeiras. O FastAPI/serviços e os jobs precisam verificar existência do lugar/usuário, reconciliar identidades e tratar exclusões e alterações.

4. Dicionário de documentos

Nas tabelas seguintes, “Obrigatório” corresponde estritamente ao array `required` do validador BSON/JSON Schema do v3. Um subdocumento não obrigatório pode estar ausente; campos `null` são permitidos apenas quando o tipo listado o admite. A opção padrão do MongoDB permite propriedades extras se `additionalProperties: false` não for declarada.

4.1. places — lugares e equipamentos culturais

Coleção principal da descoberta: concentra identidade, categorização, endereço, ponto no mapa, informações de visita, provedores e qualidade. O v3 exige apenas `_id`, `name`, `source_keys` e `sources` no nível superior; os demais atributos são opcionais no validador, embora a aplicação possa exigir mais para publicar.

Campo / grupo	Tipo no esquema	Regras / finalidade
_id *	string	Chave própria estável. Ex.: `mapas_saas:space:1`.
name *	string	Nome do lugar, com mínimo de 1 caractere.
type	string | null	Categoria normalizada do SPAços; pode faltar.
categories	array<string>	Taxonomia de busca e filtros.
source_category	string | null	Categoria/tipo original fornecido pela API.
description	object	`short`, `long`: string | null.
address	object	`full`, `postal_code`, `neighborhood`, `municipality`: string | null.
location	GeoJSON Point | null	`type="Point"`, `coordinates=[longitude, latitude]`, dois valores numéricos.
territory	object	`district`, `subprefecture`, `region`: string | null.
administration	object	`level`: esfera administrativa declarada (string | null).
contacts	object	`phone`, `website`: string | null.
public_access	boolean | null	Validação de acesso presencial para visitante, diferente de metadado original da API.
source_public_flag	boolean | null	Valor do campo `public` da fonte, mantido sem reinterpretar sua semântica.
source_status	integer | null	Status original da fonte; não equivale à publicação interna.
publication_status	enum	`review`, `published`, `hidden`, `archived`. Opcional no validador v3.
opening_hours	object	`text`: string|null; `verified`: boolean.
admission	object	`is_free`: boolean|null; `price_min/max`: número|null; `price_text`: string|null; `currency`: string.
accessibility	object	`declared`: boolean|null; `physical_features`: string[]; `wheelchair`, `adapted_restroom`: boolean|null; `notes`: string|null.
visit_duration_minutes	integer | null	Estimativa do SPAços, não fornecida automaticamente pelas APIs.

Campo / grupo	Tipo no esquema	Regras / finalidade
source_keys *	array<string>	Pelo menos um identificador externo, com índice único multikey.
sources *	array<object>	Pelo menos um objeto com `provider` e `external_id` obrigatórios; `layer`, `updated_at`, `record_url` opcionais.
data_quality	object	`location_valid`, `location_method`, `needs_review`, `detail_level`, `municipality_check`, `stale_source`.

* Obrigatório no nível superior do validador. Condições de qualidade e publicação são regras de aplicação, não requeridas pelo validador atual.

4.2. events — eventos culturais

Um evento registra informações comuns (nome, classificação, descrição) e um array de ocorrências. A escolha evita pressupor que um evento acontece sempre em um só lugar e uma só data. Na amostra recebida, nenhum dos vinte eventos tinha ocorrências; todos devem ser tratados como não agendáveis.

Campo / grupo	Tipo	Regras / finalidade
_id *	string	Ex.: `mapas_saas:event:11`.
title *	string	Título do evento, mínimo 1 caractere.
source_type	string | null	Tipo informado pelo Mapas Culturais.
source_status	integer | null	Status original da API.
description	object	`short`, `long`: string | null.
categories	array<string>	Taxonomia de categorias do SPAços.
age_rating	string | null	Classificação etária original (ex.: “Livre”).
price_text	string | null	Preço textual; não converter ausência em gratuito.
sign_language	boolean | null	Tradução em Libras declarada pela fonte.
audio_description	boolean | null	Audiodescrição declarada pela fonte.
publication_status	enum	`review`, `published`, `hidden`, `archived`.
occurrences	array<object>	`external_id`, `place_id`, `source_place_id`, `schedule_text`, `rule`, `price_text`, `starts_at`, `ends_at`.
source_keys *	array<string>	Identidades externas; mínimo de um elemento.
sources *	array<object>	Proveniência; `provider` e `external_id` obrigatórios.
data_quality	object	`detail_level`, `schedulable`, `schedule_status`, `stale_source`.

Regra para roteiro Um evento só pode ser encaixado na agenda se possuir ocorrência concreta e válida, com espaço, dia e horário confirmados, e se atender aos filtros de publicação. `schedule_text` e `rule` não bastam para garantir disponibilidade sem expansão validada da regra.

4.3. territories — limites administrativos

Campo	Tipo	Regras / finalidade
_id *	string	Identidade estável do polígono importado.
kind *	enum	`district` ou `subprefecture`.
name *	string	Nome do distrito ou subprefeitura.
region	string | null	Região administrativa associada, se houver.
geometry *	GeoJSON Polygon/MultiPolygon	Geometria completa em EPSG:4326; indexada por `2dsphere`.
sources *	array<object>	Origem e identificador GeoSampa; mínimo de um elemento.

A coleção não contém lugares: armazena polígonos para associação espacial posterior. Parques municipais também são representados por polígonos na fonte, mas o v3 não os transforma automaticamente em pontos de visitação.

4.4. place_curations — atributos editoriais

Campo	Tipo	Regras / finalidade
_id *	string	Identidade do registro de curadoria.
place_id *	string	Referência a places._id; índice único.
pet_friendly	boolean | null	Declaração curada sobre aceitação de animais.
family_friendly	boolean | null	Informação curada sobre adequação a famílias.
lgbtqia_inclusion	boolean | null	Critério curado; exigir definição explícita e evidência.
hidden_gem	boolean | null	Seleção editorial de lugar “fora do óbvio”.
accessibility_notes	string | null	Informações adicionais verificadas de acessibilidade.
opening_hours_notes	string | null	Notas sobre horário e confiabilidade da informação.
evidence	array<object>	Evidências de curadoria; estrutura interna ainda aberta no v3.

Campo	Tipo	Regras / finalidade
reviewed_at	BSON date | string | null	Registro de revisão; preferir BSON Date em novos documentos.

Os critérios de inclusão, pet-friendly e adequação familiar precisam ser verificáveis, claros e não devem ser inferidos apenas do nome ou da categoria do espaço.

4.5. sync_runs — auditoria de sincronização

Campo	Tipo	Regras / finalidade
_id *	string	Identificador da execução do job.
provider *	string	Fonte monitorada (Mapas Culturais, GeoSampa etc.).
started_at *	BSON date | string | null	Data de início (o tipo aceita null, embora a propriedade seja obrigatória).
finished_at	BSON date | string | null	Data de encerramento.
status *	enum	`running`, `success`, `partial`, `error`.
imported	integer	Quantidade efetivamente gravada.
errors	array<string>	Mensagens de erro para auditoria.

4.6. users — contas

Campo	Tipo	Regras / finalidade
_id *	string	Identificador interno do usuário.
name	string | null	Nome opcional.
email *	string	E-mail; índice único. O validador não aplica formato ou normalização do e-mail.
password_hash *	string	Hash de senha; nunca persistir senha em texto puro.
preferences	object	Preferências da experiência; formato ainda aberto no v3.
created_at	BSON date | string | null	Data de criação.

4.7. itineraries — roteiros

Campo	Tipo	Regras / finalidade
_id *	string	Identificador do roteiro.
user_id *	string	Dono do roteiro: users._id.

Campo	Tipo	Regras / finalidade
title *	string	Título do roteiro.
date	string | null	Data da visita: formato ainda não restrito no schema.
preferences	object	Filtros/condições usados para criar roteiro.
stops *	array<object>	Sequência incorporada de paradas; ordem definida pelo atributo order.
stops[].place_id *	string	Referência ao lugar.
stops[].order *	int32	Posição da parada, inteira.
stops[].arrival_time	string | null	Horário planejado da chegada.
stops[].departure_time	string | null	Horário planejado da saída.
stops[].event_id	string | null	Evento opcional que motiva a parada.
created_at	BSON date | string | null	Data da criação.

A ordem deve ser única por roteiro e coerente com a lista; o schema exige cada order, mas não impede ordens repetidas. O serviço de planejamento deve validar integridade temporal, orçamento, deslocamento e disponibilidade.

4.8. user_places — favoritos e visitados

Campo	Tipo	Regras / finalidade
_id *	string	Identidade do vínculo.
user_id *	string	Referência a users._id.
place_id *	string	Referência a places._id.
is_favorite *	boolean	Indica se é favorito.
is_visited *	boolean	Indica se foi visitado.
visited_at	BSON date | string | null	Data da visita, se conhecida.

Um índice único composto `(user_id, place_id)` impede vínculos duplicados para o mesmo usuário e lugar, sem obrigar listas extensas dentro do documento users.

5. Integração, normalização e qualidade de dados

5.1. Fontes externas

Fonte	O que alimenta	Persistência / tratamento
Mapas Culturais (instância SaaS)	Campos detalhados de espaços e eventos; horários, acessibilidade e regras de ocorrências são opcionais.	Adapter → normalização → places/events. URL que respondeu em teste: `https://saas.mapas.cultura.gov.br`.

Fonte	O que alimenta	Persistência / tratamento
GeoSampa WFS	Equipamentos, museus, bibliotecas, teatros, monumentos e limites administrativos.	GetFeature / EPSG:4326 → places/territories. Esquema de campos varia por camada.
SPTrans	Dados de transporte e previsão de chegada.	Consulta sob demanda, cache curto; não modelado em coleção permanente v3.
Open-Meteo	Dados de clima previstos.	Consulta sob demanda, cache temporário; não modelado em coleção permanente v3.

5.2. Diagnóstico real das amostras

A coleta do Mapas Culturais retornou 20 espaços e 20 eventos detalhados. Não é uma amostra aleatória de São Paulo: são as primeiras entradas de uma instância de cobertura nacional. O GeoSampa foi validado por GetCapabilities e amostras WFS de nove camadas (5 registros por camada), incluindo as cinco camadas de equipamentos.

Critério	Espaços Mapas (n=20)	Eventos Mapas (n=20)
Nome	20 preenchidos	20 preenchidos
Endereço / programação	Endereço 19/20; horário textual 3/20	Ocorrências 0/20
Descrição longa	14/20	13/20
Acessibilidade	Declaração 4/20; recursos físicos 2/20	Libras 4/20; audiodescrição 4/20
Preço	Sem preço estruturado na seleção de espaços	0/20 com preço
Localização	4/20 com coordenadas (0,0); apenas 2 confirmados em SP na triagem conservadora	Sem ocorrências que vinculem local + data
Atualidade	Grande parte do conjunto é antiga	Todos atualizados em 2016–2017

Nas 25 amostras das cinco camadas GeoSampa de equipamentos: nome 25/25, endereço 25/25, esfera administrativa 25/25, telefone 18/25, CEP 16/25 e horário de funcionamento 0/25; atributos estruturados de preço e acessibilidade não foram identificados nessas camadas. O WFS confirmou 814 registros brutos nessas cinco camadas somadas, não necessariamente 814 lugares únicos.

5.3. Regras de transformação

- Não publicar espaços fora do município alvo: validar município declarado/endereço e coordenadas; o retângulo de São Paulo sozinho é insuficiente para distinguir municípios vizinhos.
- Descartar coordenadas (0,0) como localização utilizável; conservar o registro para revisão quando pertinente.
- Converter GeoSampa para WGS84 EPSG:4326 e armazenar coordenadas na ordem longitude, latitude; polígonos vão a territories, pontos vão a places.
- Preservar categorias originais em source_category/source_type e definir taxonomias internas em categories sem inventar classificação por similaridade.
- Separar status da API, metadado `public` e status editorial SPAços; são conceitos diferentes.

- Preservar dados de origem e identificadores em sources[]/source_keys[]; não mesclar automaticamente lugares com nomes apenas semelhantes.
- Preservar regras de eventos mas não inferir datas de visita sem expansão, timezone e verificação de recorrência.
- Tratar ausências como null e anotar origem/qualidade; não afirmar “grátis”, “acessível” ou “aberto” por omissão.

6. Validadores JSON Schema e índices

6.1. Como os schemas funcionam

O script `mongo/01_schema_and_indexes.js` do baseline v3 usa `db.createCollection({validator: {$jsonSchema: ...}, validationLevel: "strict", validationAction: "error"})`; para coleções existentes, usa `collMod`. São tipos BSON (bsonType), com `required`, `enum`, restrições locais de array e subdocumentos. O schema protege contra tipos errados, mas não garante correção histórica, geográfica ou editorial dos registros.

```
// Padrão do script mongosh v3 (resumo ilustrativo)
db.createCollection('places', {
  validator: {$jsonSchema: {
    bsonType: 'object',
    required: ['_id','name','source_keys','sources'],
    properties: {
      _id: {bsonType:'string'},
      name: {bsonType:'string',minLength:1},
      publication_status: {enum:['review','published','hidden','archived']},
      location: {anyOf:[/* GeoJSON Point */, {bsonType:'null'}]}
    }
  }},
  validationLevel:'strict', validationAction:'error'
});
```

Limite formal O validador v3 não define `additionalProperties: false`, não verifica existência de referências e não garante que a dupla longitude/latitude caia em São Paulo. A restrição de qualidade deve ser implementada no importador e no serviço.

6.2. Índices criados no baseline v3

Coleção	Índice / chave	Finalidade
places	`location: "2dsphere"`	Proximidade e filtros geográficos.
places	`source_keys: 1` (unique)	Evitar que a mesma identidade externa esteja em dois documentos.
places	`publication_status: 1, categories: 1, territory.district: 1`	Filtros de mapa e status de exibição.
places	`address.municipality: 1`	Recorte por município.
events	`source_keys: 1` (unique)	Identidade externa de evento.
events	`publication_status: 1, data_quality.schedulable: 1`	Agenda publicável e validada.
events	`occurrences.place_id: 1`	Programação por espaço cultural.
territories	`geometry: "2dsphere"`	Associação e operações geoespaciais.

Coleção	Índice / chave	Finalidade
territories	`kind: 1, name: 1`	Busca de distrito/subprefeitura.
place_curations	`place_id: 1` (unique)	No máximo uma curadoria por lugar.
sync_runs	`provider: 1, started_at: -1`	Auditoria recente por fonte.
users	`email: 1` (unique)	Identidade de login.
itineraries	`user_id: 1, date: 1`	Roteiros pessoais por data.
user_places	`user_id: 1, place_id: 1` (unique)	Evitar vínculos duplicados.

7. Exemplos de documentos e consultas

7.1. Documento de lugar (exemplo ilustrativo)

```
{
  "_id": "mapas_saas:space:1",
  "name": "Espaço de exemplo",
  "type": null,
  "categories": [],
  "address": {"full": "Endereço ilustrativo", "municipality": "São Paulo"},
  "location": {"type": "Point", "coordinates": [-46.70, -23.54]},
  "publication_status": "review",
  "opening_hours": {"text": null, "verified": false},
  "admission": {"is_free": null, "price_min": null,
                "price_max": null, "price_text": null, "currency": "BRL"},
  "accessibility": {"declared": null, "physical_features": [],
                    "wheelchair": null, "adapted_restroom": null,
                    "notes": null},
  "source_keys": ["mapas_saas:space:1"],
  "sources": [{"provider": "mapas_saas", "external_id": "1"}],
  "data_quality": {"location_valid": true, "needs_review": true,
                   "detail_level": "full", "stale_source": true}
}
```

É um exemplo de estrutura, não uma reprodução fiel de um lugar da base. Os dados reais importados devem ser obtidos da sincronização e mantidos em revisão quando a fonte estiver antiga ou incompleta.

7.2. Evento e roteiro (exemplos ilustrativos)

```
// events: cada ocorrência pode apontar para lugar diferente
{
  "_id":"mapas_saas:event:900", "title":"Oficina ilustrativa",
  "source_keys":["mapas_saas:event:900"],
  "sources":[{"provider":"mapas_saas","external_id":"900"}],
  "publication_status":"review",
  "occurrences":[{
    "external_id":"123", "place_id":"mapas_saas:space:1",
    "source_place_id":"1", "schedule_text":null,
    "rule":null, "price_text":null,
    "starts_at":null, "ends_at":null
  }],
  "data_quality":{"schedulable":false,
                  "schedule_status":"unverified_rule"}
}

// itineraries: paradas embutidas e ordenadas
{
  "_id":"route_001", "user_id":"user_001",
```

```
  "title":"Passeio cultural", "date":"2026-11-15",
  "preferences":{"transport":"walking"},
  "stops":[{"place_id":"mapas_saas:space:1","order":1,
           "arrival_time":"10:00","departure_time":"11:30",
           "event_id":null}]
}
```

7.3. Consultas usuais no mongosh

```
use spacos
show collections

// Quantidade por coleção
 db.places.countDocuments({})
 db.territories.countDocuments({})

// Publicação aprovada + localização conhecida
 db.places.find({
   publication_status: 'published',
   location: {$type: 'object'}
 }, {name:1,location:1,categories:1}).limit(10)

// Eventos de fato agendáveis e publicados
 db.events.find({publication_status:'published',
               'data_quality.schedulable':true})

// Curadorias pendentes e origem
 db.places.find({publication_status:'review'},
             {name:1,sources:1,data_quality:1}).limit(20)

// Roteiros de um usuário
 db.itineraries.find({user_id:'user_001'}).sort({date:1})
```

8. Operação e importação no Windows

Para desenvolvimento local, o projeto usa MongoDB Community Server, MongoDB Shell (mongosh) e MongoDB Database Tools (mongoimport), que pode exigir instalação e inclusão no PATH separadamente do servidor. Executar os comandos a seguir no PowerShell, na pasta extraída do kit de banco v3.

```
# PowerShell, com serviço MongoDB em execução
mongosh "mongodb://127.0.0.1:27017/spacos" .\mongo\01_schema_and_indexes.js

# Importações de amostras locais, em banco novo ou controlado
mongoimport --uri="mongodb://127.0.0.1:27017/spacos" `
  --collection=places --type=json `
  --file=.\data\geosampa_places_sample.ndjson `
  --mode=upsert --upsertFields=_id

mongoimport --uri="mongodb://127.0.0.1:27017/spacos" `
  --collection=places --type=json `
  --file=.\data\places_mapas_sp_review.ndjson `
  --mode=upsert --upsertFields=_id

mongoimport --uri="mongodb://127.0.0.1:27017/spacos" `
  --collection=territories --type=json `
  --file=.\data\geosampa_territories_sample.ndjson `
  --mode=upsert --upsertFields=_id
```

Estes nomes de arquivo correspondem ao kit de dados v3 descrito anteriormente e não estão anexados a este documento. Antes de qualquer importação em banco já utilizado, conferir os lotes, realizar backup e verificar o comportamento de substituição/atualização com --mode=upsert.

8.1. Operação segura e rastreabilidade

- Usar autenticação/HTTPS na implantação; no ambiente local, não expor a porta 27017 à rede pública.
- Guardar credenciais de aplicação em variáveis de ambiente, nunca no repositório.
- Persistir somente password_hash com algoritmo apropriado (ex.: Argon2 ou bcrypt), nunca senha original.
- Aplicar controle de acesso à leitura/edição de users, user_places e itineraries no backend.
- Registrar sync_runs com início, término, status, quantidades e erros; não confundir sucesso da requisição HTTP com qualidade editorial da fonte.
- Planejar rotina de backup e teste de restauração; importação por upsert pode sobrescrever alterações feitas fora do fluxo de curadoria.

9. Restrições e decisões ainda pendentes

Ponto	Situação atual / ação recomendada
Publicação GeoSampa	Amostras antigas GeoSampa produzidas antes do v3 não possuem `publication_status`. Migrar para `review` antes de esperar que apareçam em consultas `published`.
Eventos futuros	A amostra Mapas possui 0/20 ocorrências. Exige busca direcionada a eventos futuros e parser confiável de calendário.
Atualidade	Datas antigas requerem alerta/curadoria e regra de revalidação; a API não garante que todo espaço existe ou funciona.
Identidade entre APIs	Mesmo espaço pode ocorrer em fontes distintas. Nenhuma fusão cross-provider foi confirmada; definir regras de correspondência assistida.
Normalização territorial	Confirmar município por polígono administrativo e/ou fonte confiável, não apenas bounding box.
Horários e preços	Dados frequentemente ausentes ou texto livre. Não são adequados para prometer roteiro aberto/viável sem confirmação.
JSON Schema	Alguns grupos são livres (`preferences`, `evidence`, `occurrences.rule`); formalizar formatos ao consolidar regras de negócio.
Datas	O v3 aceita BSON Date, string ou null em diversos campos. Padronizar novos dados em BSON Date, com UTC e timezone de negócio explícito.
Integridade referencial	Implementar validação de place_id/user_id no serviço e estratégia para exclusão de usuário, lugar ou evento.
Banco real	A verificação original foi feita com testes locais/validação de documentos; não há comprovação de implantação do script num servidor MongoDB de produção.
Catálogo abrangente	GeoSampa informou contagens por camada, mas as importações anteriores continham só cinco registros por camada. Importação integral demanda paginação e reconciliação.

10. Referência rápida para implementação

O fluxo funcional de consulta deve ser: React/PWA → FastAPI (router) → serviço de domínio → repositório MongoDB. Sincronizações periódicas executam Mapas Culturais/GeoSampa → adaptadores → normalização/validação → possível reconciliação → persistência; os serviços de curadoria atualizam informações separadas. Para gerar roteiros, o planejador consulta lugares aprovados e tenta garantir tempo/orçamento/horário, anexando informações efêmeras de transporte e clima quando disponíveis.

10.1. Checklist mínimo de aceite

- Criar oito coleções e 14 índices definidos no script v3 em um banco novo; conferir `getIndexes()`.
- Importar registros de exemplo sem violar validadores; confirmar contagens e geometria WGS84.
- Confirmar que `places` inclui apenas lugares aprovados quando o filtro pede `publication_status: published`.
- Não exibir eventos históricos ou sem ocorrência como programação futura.
- Testar consistência de user_id/place_id/event_id nos serviços de aplicação.
- Simular indisponibilidade das APIs externas e verificar que mapa e busca continuam usando dados sincronizados.
- Verificar deduplicação e preservação de curadoria em duas sincronizações sucessivas.
- Revisar política de LGPD, exclusão de conta e eliminação de informações pessoais associadas.

11. Documentos de origem e rastreabilidade

Esta especificação foi compilada a partir dos seguintes materiais do projeto, preservando as decisões do baseline v3 e distinguindo observação empírica de proposta técnica:

Fonte	Aspectos utilizados
Arquitetura.pdf — “SPAços: Definição da Arquitetura Inicial”, sobretudo seções 3 a 5	ASRs, stack FastAPI/React/MongoDB, integrações, curadoria, segurança, modelo lógico e roteiro.
DescricaoProjeto.pdf	Propósito, principais funcionalidades e estratégia de APIs.
space_find_detailed.json	20 espaços retornados pelo Mapas Culturais; propriedades, completude e problemas de coordenadas.
event_find_detailed.json	20 eventos retornados; descrições, classificação, acessibilidade declarada, ausência de ocorrências e preço.
followup_live.zip	GetFeature/DescribeFeatureType de nove camadas GeoSampa, schemas Mapas Culturais e relatório das consultas.
Baseline MongoDB v3 discutido no projeto	Oito coleções, validações BSON, 14 índices, nomes de atributos, importação e limitações.

Critério editorial: exemplos de documentos foram identificados como ilustrativos; números indicam amostras e nunca são apresentados como estimativas da população de lugares de São Paulo.

FIM DA ESPECIFICAÇÃO • SPAços